

MANUAL DEL CODIGO FUENTE

Elaborado por Brayan Alexander Barco Cardenas

Asignatura: Sistemas Operativos

Docente de asignatura: Miguel Fabian Robles Angarita

Universidad Francisco de Paula Santander

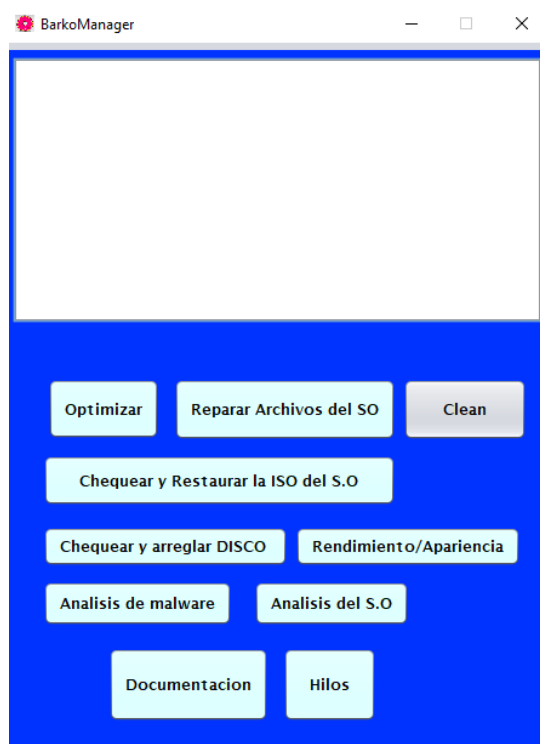
Proyecto: Barcodehub/BarkoManager

BarkoManager

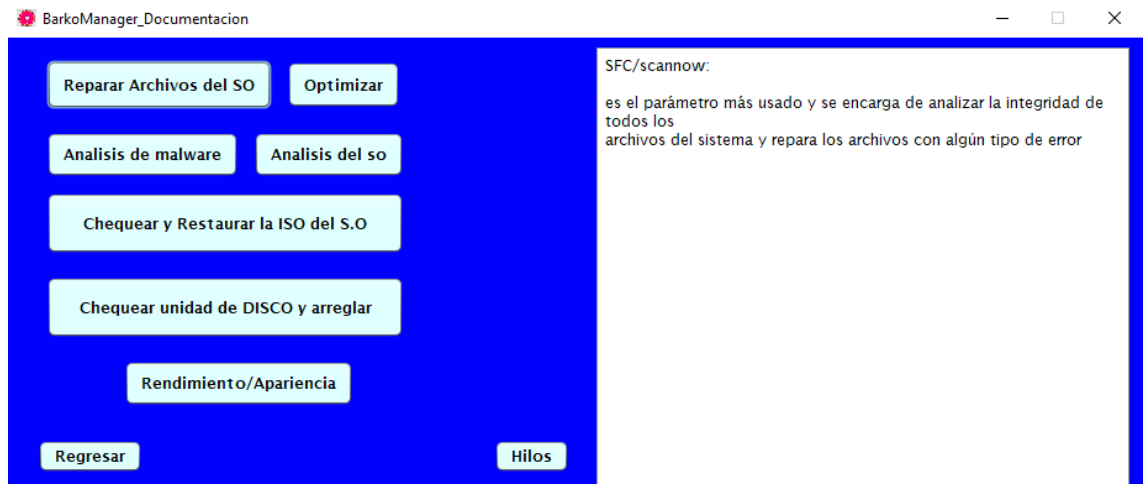
Organización del Proyecto

Paquete Calculadora:

1. **Clase BarkoManager:** Esta clase contiene una serie de métodos que se invocan cuando se presionan varios botones en la interfaz de usuario. Cada método realiza una tarea específica, como limpiar la caché de Internet y los archivos temporales, ejecutar un análisis de malware, comprobar y reparar la imagen del sistema operativo, entre otros.

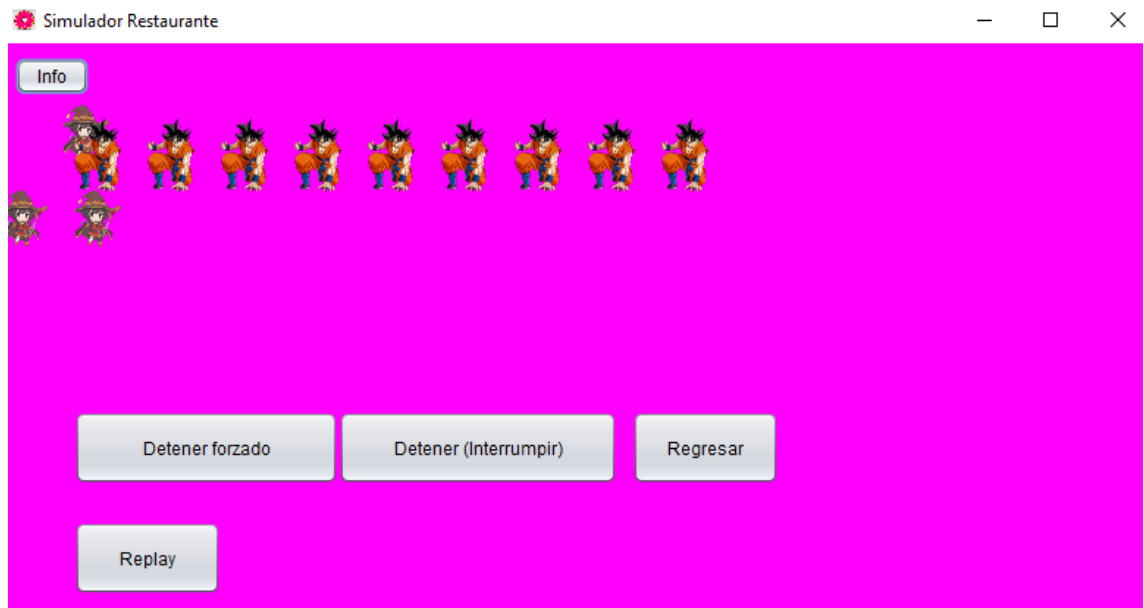


2. **Clase documentacion:** Esta clase proporciona la documentación para cada uno de los comandos que se pueden ejecutar a través de la interfaz de usuario. Cada método en esta clase proporciona una descripción detallada de lo que hace un comando específico.



Paquete Restaurant:

1. **Clase RestauranteApp:** Esta es la clase principal que inicia la aplicación. Crea una instancia de Restaurante e inicia el proceso.
2. **Clase Cliente:** Esta clase representa a un cliente en el restaurante. Cada cliente tiene un nombre y un índice que se utilizan para identificarlo.
3. **Clase ClienteGUI:** Esta clase es una extensión de JLabel y representa la interfaz gráfica de usuario para un cliente. Tiene métodos para mover la etiqueta a una ubicación específica y para hacerla invisible.
4. **Clase RestauranteUI:** Esta clase es una extensión de JFrame y representa la interfaz gráfica de usuario para el restaurante. Tiene un método para mostrar mensajes en un JTextArea.
5. **Clase MeseroGUI:** Esta clase es una extensión de JLabel y representa la interfaz gráfica de usuario para un mesero. Tiene un método para mover la etiqueta a una ubicación específica.
6. **Clase Mesero:** Esta clase implementa la interfaz Runnable y representa a un mesero en el restaurante. Cada mesero tiene un nombre, un índice y una referencia al restaurante en el que trabaja. Los meseros pueden atender a los clientes y moverse por el restaurante.
7. **Clase Restaurante:** Esta es la clase más compleja y representa al restaurante. Tiene una cola de clientes que esperan ser atendidos y un array de hilos para los meseros. También tiene una interfaz gráfica de usuario que muestra a los clientes y a los meseros. Los clientes se agregan a la cola y los meseros los atienden.



Comandos CMD utilizados en la calculadora:

El código utiliza varios comandos del sistema operativo para realizar diversas tareas:

1. **ipconfig /flushdns**: Este comando se utiliza para limpiar la caché del DNS en tu sistema. Limpiar la caché del DNS puede ayudar a resolver problemas de conexión a Internet.
2. **del .tmp /s /f /q, del .log /s /f /q, del c:\windows\temp\ /s /f /q, del %temp% /s /f /q, del c:\windows\prefetch\ /s /f /q**: Estos comandos se utilizan para eliminar varios tipos de archivos temporales de tu sistema. Los archivos temporales pueden ocupar espacio en el disco y ralentizar tu sistema, por lo que es una buena idea eliminarlos de vez en cuando.
3. **sfc /scannow**: Este comando se utiliza para escanear los archivos del sistema en busca de errores y, si es posible, repararlos. Esto puede ayudar a resolver problemas con Windows y mejorar el rendimiento del sistema.
4. **DISM /Online /Cleanup-Image /CheckHealth, DISM /Online /Cleanup-Image /ScanHealth, DISM /Online /Cleanup-Image /RestoreHealth**: Estos comandos se utilizan para comprobar la salud de la imagen del sistema operativo y, si es necesario, repararla. Esto puede ser útil si estás experimentando problemas con Windows.
5. **chkdsk C: /F /R**: Este comando se utiliza para comprobar el disco duro en busca de errores y, si es posible, repararlos. Esto puede ayudar a prevenir la pérdida de datos y mejorar el rendimiento del disco duro.
6. **winsat formal -v**: Este comando se utiliza para realizar una evaluación detallada del rendimiento de tu sistema. Puede proporcionarte información útil sobre cómo mejorar el rendimiento de tu sistema.
7. **mrt**: Este comando se utiliza para ejecutar la Herramienta de eliminación de software malintencionado de Microsoft. Esta herramienta puede ayudarte a eliminar el software malintencionado de tu sistema.

8. **SystemPropertiesPerformance.exe**: Comando con el cual carga el apartado para priorizar ya sea el rendimiento o la apariencia de Windows, lo normal es en default

BarkoManager

Calculadora: Administrar y optimizar el rendimiento en Windows

Aplicacion desktop para administrar y mejorar el rendimiento en Windows 10 y Windows 11.

La **concurrency** en este código se logra a través del uso de **SwingWorker**, que permite ejecutar tareas en segundo plano en un hilo separado, de esta manera se logra que en tiempo real se obtenga la información en pantalla de lo que está arrojando el **CMD** y no tener que esperar hasta el final. Hay dos métodos principales que se sobrescriben en **SwingWorker**:

doInBackground(): Este método se ejecuta en un hilo separado y es donde se realiza la tarea en segundo plano. En este caso, la tarea es ejecutar un comando del sistema operativo y leer su salida.

process(): Este método se ejecuta en el hilo de despacho de eventos de Swing y se utiliza para actualizar la interfaz gráfica de usuario con los resultados de la tarea en segundo plano. En este caso, se utiliza para agregar la salida del comando a un área de texto.

Ejemplo sencillo (GUI) para explicar la concurrency en Java:

Simulación de Restaurante

Este proyecto incluye un ejemplo sencillo de cómo implementar concurrency en Java utilizando hilos (threads) y mecanismos de sincronización. El objetivo es simular el funcionamiento de un restaurante de manera gráfica.

Funcionamiento del programa

1. **Inicio de la aplicación:** Cuando se inicia la aplicación, se crea una instancia de **Restaurante** y se llama al método **iniciar()**. Este método inicia los hilos **Mesero** y agrega los **Cliente** a la cola.
2. **Atención de los clientes:** Cada hilo **Mesero** intenta obtener un **Cliente** de la cola. Si la cola está vacía, el hilo **Mesero** espera hasta que haya un **Cliente** disponible. Una vez que un **Mesero** obtiene un **Cliente**, lo atiende. La atención del cliente implica mover al **Mesero** a la ubicación del **Cliente**, esperar un tiempo aleatorio para simular la atención, y luego mover al **Mesero** de vuelta a su posición original.
3. **Interfaz gráfica de usuario:** La interfaz gráfica de usuario muestra a los **Cliente** y a los **Mesero**. Cuando un **Cliente** es atendido, se elimina de la interfaz gráfica de usuario. Cuando todos los **Cliente** han sido atendidos, se muestra un mensaje y se interrumpen los hilos **Mesero**.

4. **Interrupción de los hilos:** Los hilos Mesero pueden ser interrumpidos de dos maneras. Una forma es cuando el numero de Clientes sobrepasa el limite de atendidos (1000). La otra forma es cuando se pulsa el botón de detener. Cuando un hilo Mesero es interrumpido, deja de atender a los Cliente y cambia su estado a interrumpido, después de un poco, termina su ejecución.
5. **Detener los meseros:** Cuando se pulsa el botón de detener, se interrumpen todos los hilos Mesero y se cierra la interfaz gráfica de usuario del restaurante.

Uso de la concurrencia

1. **Hilos:** En Java, la concurrencia se logra a través del uso de hilos. Un hilo es una unidad de ejecución dentro de un proceso. En este código, cada Mesero se ejecuta en su propio hilo. Esto permite que varios Mesero atiendan a los Cliente al mismo tiempo, al igual que en un restaurante real.
2. **Sincronización:** La sincronización es un aspecto importante de la programación concurrente. En este código, se utiliza un objeto de bloqueo (lock) para asegurar que sólo un hilo Mesero pueda acceder a la cola de Cliente en un momento dado. Esto evita las **condiciones de carrera**, que pueden ocurrir cuando dos o más hilos intentan modificar una variable compartida al mismo tiempo.
3. **Comunicación entre hilos:** Los hilos Mesero y Cliente se comunican entre sí a través de la cola de Cliente. Cuando un Cliente es atendido, se elimina de la cola. Si la cola está vacía, los hilos Mesero esperan hasta que haya un Cliente disponible.
4. **Interrupción de hilos:** Los hilos Mesero pueden ser interrumpidos, lo que les permite terminar su ejecución antes de tiempo. Esto se utiliza en este código para detener a los Mesero una vez que todos los Cliente han sido atendidos, o cuando se pulsa el botón de detener.
5. **Espera activa:** La espera activa se utiliza en este código para simular el tiempo que tarda un Mesero en atender a un Cliente. Esto se logra haciendo que el hilo Mesero duerma durante un tiempo aleatorio.

Requisitos de software y hardware

Para ejecutar este programa, necesitarás:

Software:

1. Tener Java instalado y configurado. Necesitarás tener instalado el JDK para compilar y ejecutar el programa. La versión recomendada es la JDK 8 o superior, además de agregarla al Path dentro de las variables de entorno.
2. Un entorno de desarrollo integrado (IDE) para Java, como IntelliJ IDEA o Eclipse, preferiblemente NetBeans. Aunque no es estrictamente

necesario, un IDE puede facilitar la escritura, compilación y depuración del código.

Hardware:

1. Como este es un programa basado en texto con una carga de procesamiento relativamente baja, los requisitos de hardware son mínimos. Cualquier computadora moderna debería ser capaz de ejecutar este programa sin problemas.

Cómo contribuir al proyecto:

Este proyecto está licenciado bajo la Apache License 2.0, lo que significa que cualquiera es libre de utilizar, modificar y distribuir el código fuente. Si estás interesado en contribuir al proyecto, aquí tienes algunos pasos que puedes seguir:

1. **Fork el repositorio:** Haz un fork del repositorio del proyecto en tu propia cuenta de GitHub. Esto te dará tu propia copia del código fuente que puedes modificar.
2. **Clona el repositorio:** Clona el repositorio a tu máquina local para que puedas empezar a trabajar en él.
3. **Crea una rama:** Crea una nueva rama en la que realizarás tus cambios. Esto ayuda a mantener separados los diferentes conjuntos de cambios y facilita la revisión de las contribuciones.
4. **Realiza tus cambios:** Haz los cambios que quieras en el código fuente.
5. **Compromete tus cambios:** Una vez que hayas terminado de hacer tus cambios, compromételos en tu rama.
6. **Envía una solicitud de pull:** Envía una solicitud de pull al repositorio original para que los mantenedores del proyecto puedan revisar y, con suerte, fusionar tus cambios en el código fuente principal.

Algunos Cambios para remarcar:

En el restaurante caben 10 clientes, los cuales son atendidos y luego de un segundo llega un nuevo cliente a ocupar su lugar. Ahora hay un límite de Clientes que llegan al Restaurante, una vez todos han sido atendidos, el Restaurante cierra sus puertas

Se implementa un botón de Re-play que detiene la ejecución y vuelve a empezar; además de un botón de Regresar para volver a la pantalla del BarkoManager y el botón detener se divide en 2, uno de detención forzada y detención por medio de la interrupción a los hilos, ahora te dice cuántos clientes alcanzaron a ser atendidos. Se integra un botón info para explicar el detalle de la detención e interrupción de los hilos.

Lo que se escriba en el textArea de la Calculadora ahora se envía a la consola.

